Linting

Linting for UI extensions

An overview of ESLint shareable configurations for building UI extensions on HubSpot with React.

For security and reliability, UI extensions run in a sandboxed web worker environment with restricted APIs. This can cause issues or unexpected behavior when developing UI extensions, especially if you're not fully familiar with the UI extensions environment.

The `@hubspot/eslint-config-ui-extensions` package provides ESLint rules specifically designed for HubSpot UI extensions, and catches common issues like using unavailable browser APIs, incorrect imports, and other patterns that won't work in the sandboxed web worker context.

It's recommended to set up ESLint with this configuration when starting a new UI extensions project, but can be implemented at any time.

Installation

Upgrading from v0.x? See the **migration guide** for step-by-step instructions.

Install **ESLint** 9+ alongside this package:

```
npm i --save-dev eslint@^9 @hubspot/eslint-config-ui-extens
```

Basic usage

Create an `eslint.config.js` file in the root of your project:

HubSpot Developers

```
  // Your project-specific overrides
];
```

Make sure your `package.json` has `"type": "module"` set, then add a lint script:

```
{
  "type": "module",
  "scripts": {
    "lint": "eslint ."
  }
}
```

Run `npm run lint` to check for linting issues.

Recommended setup

The basic config above only includes HubSpot UI extensions-specific rules. For a production-ready setup, it's recommended that you add standard ESLint rules, TypeScript support, React Hooks linting, Prettier compatibility, and unused import detection.

Install additional dependencies

```
npm i --save-dev @eslint/js typescript-eslint eslint-plugin
```

Full config

HubSpot Developers

```
import reactHooks from 'eslint-plugin-react-hooks';
import unusedImports from 'eslint-plugin-unused-imports';
import prettier from 'eslint-config-prettier';
import { config as uiExtensionsConfig } from '@hubspot/eslint-config-ui-extensi

export default defineConfig([
  // Standard ESLint recommended rules
  js.configs.recommended,

  // TypeScript support with stylistic rules
  tseslint.configs.recommended,
  tseslint.configs.stylistic,

  // React recommended rules + JSX runtime support (React 17+)
  react.configs.flat.recommended,
  react.configs.flat['jsx-runtime'],
  {
    settings: {
      react: { version: 'detect' },
    },
  },

  // HubSpot UI extensions rules
  ...uiExtensionsConfig,

  // React Hooks rules
  reactHooks.configs.flat['recommended-latest']

  // Unused imports detection (auto-fixable with --fix)
  {
    plugins: {
      'unused-imports': unusedImports,
    },
    rules: {
      'no-unused-vars': 'off',
      '@typescript-eslint/no-unused-vars': 'off',
      'unused-imports/no-unused-imports': 'error',
      'unused-imports/no-unused-vars': [
        'warn',
```

HubSpot Developers

```
        >
        },
      ],
      'react/prop-types': 'off',
    },
  },

  // Prettier compatibility (must be last to override other configs)
  prettier,
]);
```

What each addition provides

Package	Purpose
`@eslint/js`	ESLint's built-in recommended rules for catching common JavaScript issues
`typescript-eslint`	TypeScript parsing and linting rules (`stylistic` adds consistent code style enforcement)
`eslint-plugin-react`	React-specific rules; `jsx-runtime` config is for React 17+ (no need to import React in every file)
`eslint-plugin-react-hooks`	Enforces **Rules of Hooks** and verifies dependency arrays
`eslint-config-prettier`	Disables ESLint rules that conflict with Prettier formatting
`eslint-plugin-unused-imports`	Auto-fixable detection and removal of unused imports

JavaScript-only projects

If you're not using TypeScript, you can simplify the config:

HubSpot Developers

```
import unusedImports from 'eslint-plugin-unused-imports';
import prettier from 'eslint-config-prettier';
import { config as uiExtensionsConfig } from '@hubspot/eslint-config-ui-extensi

export default defineConfig([
  js.configs.recommended,
  react.configs.flat.recommended,
  react.configs.flat['jsx-runtime'],
  {
    settings: {
      react: { version: 'detect' },
    },
  },
  ...uiExtensionsConfig,
  reactHooks.configs.flat['recommended-latest'],
  {
    plugins: {
      'unused-imports': unusedImports,
    },
    rules: {
      'no-unused-vars': 'off',
      'unused-imports/no-unused-imports': 'error',
      'unused-imports/no-unused-vars': [
        'warn',
        {
          vars: 'all',
          varsIgnorePattern: '^_',
          args: 'after-used',
          argsIgnorePattern: '^_',
        },
      ],
      'react/prop-types': 'off',
    },
  },
  prettier,
]);
```

HubSpot Developers

›

Name	Description
no-browser-dialogs	Prevent usage of native browser dialog APIs (alert, confirm, and prompt) in UI extensions.
no-browser-storage	Prevent usage of browser storage APIs (`localStorage` and `sessionStorage`) in UI extensions, which run in a sandboxed web worker environment where these APIs are not available.
no-console	Prevent usage of console methods in UI extensions in favor of the SDK logger utility.
no-dom-access	Prevent access to the DOM (`document` object) in UI extensions, which run in a sandboxed web worker without DOM access.
no-html-elements	Disallow usage of unsupported HTML elements in UI extensions.
no-invalid-extension-point-imports	Prevent importing components from unsupported extension points.
no-invalid-image-src	Only allow valid image URLs in the src attribute of Image components from @hubspot/ui-extensions.
no-native-http	Prevent usage of native browser HTTP APIs (fetch and XMLHttpRequest) in UI extensions.
no-parent-imports	Prevent importing files from outside the extension point root directory. This rule can be automatically fixed by the `--fix` **CLI option**.
no-restricted-globals	Prevent usage of browser globals that are not available in the sandboxed web worker environment.

Last modified on February 27, 2026

Was this page helpful? Yes No